CSC 2400 VRP

Project Overview

For our CSC 2400 term project, our team is looking at the Capacitated Vehicle Routing Problem. The goal of the CRVP is to determine efficient routes for a group of vehicles that all must deliver to multiple customers. Every vehicle begins and ends at a central spot, every customer must be visited, and each customer has a amount that must be reached. The other issue is that every vehicle has a limited capacity and must be under that amount.

Our project will compare three different approaches to solving this problem, the Nearest Neighbor Heuristic, the Clarke Wright Savings algorithm and the Harmony Search. We chose these three because they are three different ways to solve the same problem. Nearest neighbor is a greedy algorithm that gives us a useful baseline. Clarke Wright provides a more distinct option that trys to combine the routes based on the distance saved. Harmony Search tries to identify the best route but sometimes could produce better routes at the cost of extra time.

The main goal of this project is too identify which of these different routes will be the best it terms of total distance, num of routes and the ability to handle more and more customers.

The github repo has been organized into separate folders for code, datasets, results, and reports. The code folder contains the .py files and the experiment runner. The data folder contains the customer information used throughout testing. The results folder contains the csv files with the results. The tables and graphs folder will hold the visual comparisons for the final draft and the reports folder contains this file and the final project documents. Else the readme explains the everything needed to follow and understand the project.

The main .py folder and code done for this checkpoint is the Nearest Neighbor baseline file. The algorithm begins at customer 0, which for our problem it represents the depot. It creates and maintains a collection of customers that have not yet been visited and continues to select the closest unvisited customer whose demand fits within the capacity of the vehicle. When no addistion customer will fit on the vehicle then the route will end at the depot and a new route begins. The program then checks whether any individual customer demand grows the vehicle capacity. If this occurs, it will throw an error.

For the nearest neighbor the experiment runner reads the customer coordinates and demands from the csv files. It will run the algorithm and measures execution time using a performance timer. Also it calculates the final route distances, counts num of routes and saves the results in a similar csv format. I thought separating the algorithm from the

experiment runner was a good decision because it will allow the other algorithms to be added without having to rewrite the result portions.

After the first test, we created a small custom dataset containting one depot with ten customers. Each customer should have their own id and a demand. For initial testing the capacity was set to 40.

I got 4 routes from the initial set

1. 0 to 10 to 7 to 1 to 0, with a demand of 40
2. 0 to 5 to 8 to 6 to 0, with a demand of 40
3. 0 to 9 to 4 to 3 to 0, with a demand of 40
4. 0 to 2 to 0 with a demand of 15

The rest of the metric were saved into that raw_results.csv

For this checkpoint I thought it best to use a small dataset so it would be easier to follow the routes but for the final project we would use the CVRPLIB instances so that we could test the different algorithm on different sizes.

As for next steps I plan to implement the additional algorithms and prepare larger datasets so that all three algorithms could be evaluated with a central program.